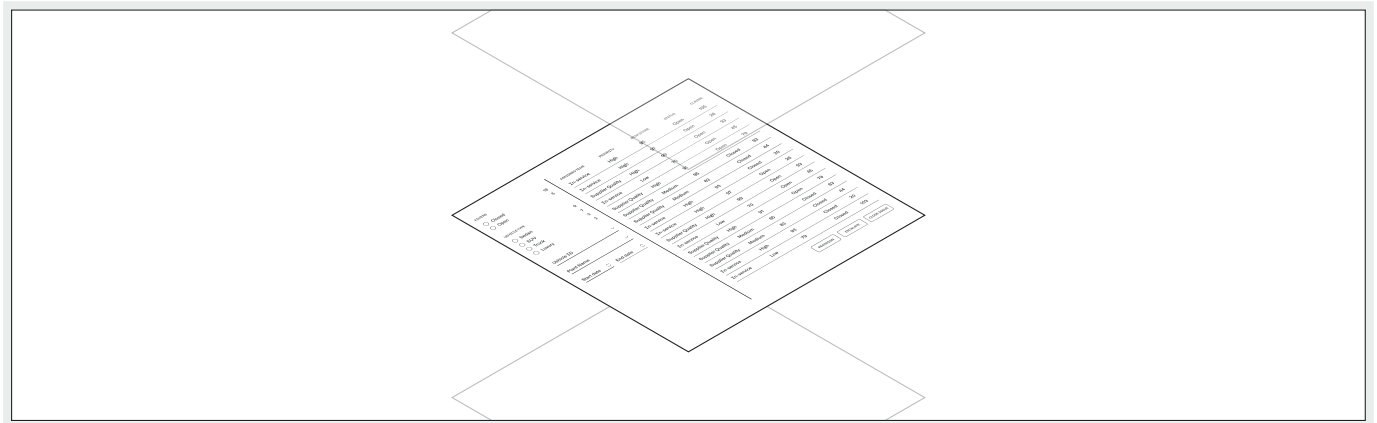


Table of Contents

1. Overview • App building • Palantir
2. Overview • App building • Palantir
3. What is an operational application? • Palantir
4. Connecting analytics to operations • Palantir
5. Conduct sentiment analysis with AIP • Palantir
6. Curating apps in Applications Portal • Palantir

Use case development / Overview



Use case development

The Palantir platform was designed to empower a diverse community of builders with a collection of powerful tools for use case development, including application building tools, workflow building tools, integrated [analytics tools](#), and [developer tools](#). Each of these leverages the power of Foundry's core security, lineage, data, and compute primitives, allowing teams to focus on delivering operational capability rather than managing infrastructure. Each tool in the Palantir platform is designed to continuously and safely enrich a consistent set of data and logical assets contained within the [Ontology](#). This enables knowledge to compound as operational workflows are scaled out across the enterprise.

i Palantir Learning portal

Understand scoping use cases for Foundry and AIP at learn.palantir.com ↗.

A API Reference ↗ building

T   platform offers several ways to build applications, suited

Send feedback

- [Workshop](#) is a no-code, object-oriented builder for operational applications, and is the fastest way to turn objects, links, and actions into an interactive workflow.
- [OSDK React applications](#) let you build completely custom user interfaces with React, powered by the [Ontology SDK \(OSDK\)](#), when you need an experience beyond what Foundry's built-in tools offer. [Pilot](#) provides a streamlined, AI-powered way to build these OSDK applications from a natural-language prompt.
- [Custom widgets](#) let technical builders extend Workshop with custom frontend code, adding tailored functionality without building a standalone application from scratch.
- [Slate](#) is a low-code, drag-and-drop builder for operational applications, interactive dashboards, and custom landing pages, with optional customization using HTML, CSS, and JavaScript.
- [Code Workspaces applications](#) let you build and publish interactive Streamlit, Dash, and Shiny® applications from third-party IDEs, backed by Foundry data and governance.

Workshop

Workshop is a flexible, object-oriented application building tool. Workshop leverages the semantic primitives (such as objects and links) and the kinetic primitives (such as actions and functions) within the [Ontology](#) to enable the rapid delivery of interactive web and mobile applications. The application building experience in Workshop empowers users to create powerful applications out of no-code, low-code, and code-based widgets. No technical expertise is required to start building with widgets and weaving objects, links, and actions together into user-driven workflows that go [far beyond dashboards](#) or passive visualizations. Meanwhile, code-based enrichment with [functions](#) can be seamlessly embedded within Workshop widgets to allow for complex interactions, cascading processes, and complex data capture. When a workflow needs functionality beyond the built-in widgets, you can extend a Workshop application with [custom widgets](#) to add tailored, code-based components without leaving Workshop.

[Learn more about Workshop.](#)

OSDK React applications

OSDK React applications let you build completely customizable user interfaces using [React](#), powered by the [Ontology SDK \(OSDK\)](#) and created through [Pilot](#). By treating Foundry as your backend, you can combine the React ecosystem with Foundry's high-scale queries, edits, and granular governance controls to securely deliver

API Reference ↗

Send feedback

bespoke applications. To build an OSDK React application without starting from code, [Pilot](#) offers a streamlined, AI-powered experience that generates the Ontology entities, design, and frontend from a natural-language prompt and guides you through deployment.

[Learn more about OSDK React applications.](#)

Custom widgets

Custom widgets allow technical builders to securely extend Workshop applications with custom frontend code. Rather than building a standalone application from scratch, you can implement functionality that is not available out of the box, such as bespoke visualizations or discipline-specific views of Ontology objects, and embed them directly within a Workshop module. You can also embed a single, full-page custom widget within Workshop to bring the flexibility of custom code to the Workshop framework.

[Learn more about custom widgets.](#)

Slate

Slate provides builders with a flexible set of tools to quickly create operational applications and interactive dashboards. Slate enables application developers to construct dynamic and responsive applications with a drag-and-drop interface, reducing development time and cost. Slate includes capabilities that are seamlessly integrated with the Foundry Ontology, but also enables developers to fully customize applications using HTML, CSS, and JavaScript.

[Learn more about Slate.](#)

Code Workspaces applications

Code Workspaces applications let you build and publish interactive web applications, using [Streamlit](#) or [Dash](#) for Python and [Shiny®](#) for R, directly from third-party IDEs in

[API Reference](#) data scientists and developers can turn analyses into operational applications on Foundry data, hosted in Foundry containers with the platform's version control, branching, and data governance built in.

Send feedback

[Learn more about Code Workspaces.](#)

Workflow building and management

The primary workflow building and management tools in the Palantir platform are [Workflow Lineage](#), [Automate](#), [Solution Designer](#), and [Use Cases](#).

Workflow Lineage

Workflow Lineage provides an interactive workspace for understanding and managing applications and their underlying processes. With Workflow Lineage, you can explore workflows and view details on objects, actions, functions, large language models, and applications. Workflow Lineage is particularly useful for application builders that are creating, debugging or maintaining workflows. The graph of provenance, deeper property and workshop widget/variable provenance, and upgrade tooling are all helpful when making changes to or extending a workflow.

Automate

Automate gives you a single entry point for setting up and executing all business automation in the platform. The Automate application allows users to define conditions and effects; conditions are checked continuously, and effects are executed automatically when the specified conditions were met.

[Learn more about Automate.](#)

Carbon

Carbon enables the configuration of tailored platform experiences, known as workspaces, for specific user groups. Carbon can provide a focused experience for less technical users that need to carry out critical operational workflows. Each Carbon workspace is a curated collection of applications and resources that can be configured to optimize a given set of operational, end-user workflows. For example, an aircraft parts maintenance workflow might consist of a Workshop application containing a dynamically updated list of parts requiring maintenance, along with Ontology-driven Actions for triaging each part, another

[API Reference](#) ↗

Send feedback

application that is used to investigate each part's maintenance issue; and a Quiver analysis showing maintenance trends over time. Carbon allows the rich tapestry of Foundry applications and analytical capabilities to be integrated into focused, operational experiences.

[Learn more about Carbon.](#)

Solution Designer

Solution Designer is an interactive tool for creating architectural representations of solutions built using the Palantir platform, including representations for first and third-party integration points, links to platform resources, on-demand access to documentation and best practices, and more.

[Learn more about Solution Designer.](#)

Use Cases

The Use Cases application allows builders to organize their work within a single operational interface. By combining the file system view with an ontology management view, developers can access a curated view focused on the work for which they are responsible.

[Learn more about the Use Cases application.](#)

Developer toolchain

The Palantir [developer toolchain](#) enables you to build your own applications on top of the Palantir platform using your own tools.

The centerpiece of the Palantir developer toolchain is the [Ontology SDK \(OSDK\)](#), generated in [Developer Console](#). The Ontology SDK is created either as an npm package for TypeScript or as a pip or Conda package for Python, and it only contains a pre-selected subset of your C API Reference ↗. It lets you access object types, apply actions to update data in the Ontology, call functions, and run AIP Logic functions for [AIP-enabled](#) enrollments. The

Developer Console also includes Ontology-specific documentation for

Send feedback

for your application. Applications use the OAuth flow as a public or confidential client to access the data.

[Learn more about Ontology SDK.](#)

You can also work directly with Foundry's [REST APIs](#) to query objects, apply actions, and call functions against the Ontology from any language or runtime for applications built entirely outside of Foundry.

NEXT
What is an operational application? →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

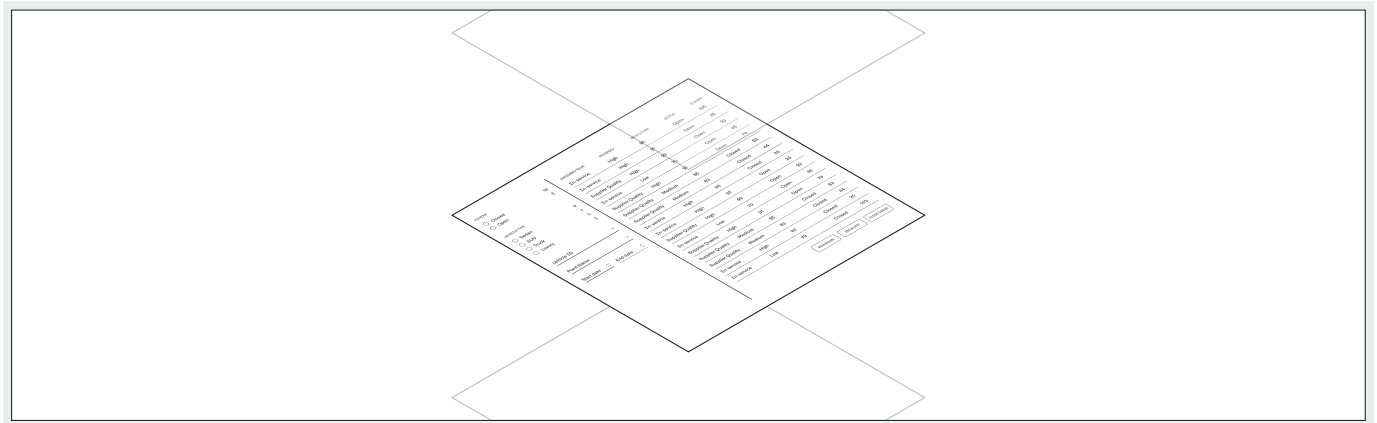
[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Use case development / Overview



Use case development

The Palantir platform was designed to empower a diverse community of builders with a collection of powerful tools for use case development, including application building tools, workflow building tools, integrated [analytics tools](#), and [developer tools](#). Each of these leverages the power of Foundry's core security, lineage, data, and compute primitives, allowing teams to focus on delivering operational capability rather than managing infrastructure. Each tool in the Palantir platform is designed to continuously and safely enrich a consistent set of data and logical assets contained within the [Ontology](#). This enables knowledge to compound as operational workflows are scaled out across the enterprise.

i Palantir Learning portal

Understand scoping use cases for Foundry and AIP at learn.palantir.com ↗.

A API Reference ↗ building

T   platform offers several ways to build applications, suited

Send feedback

- [Workshop](#) is a no-code, object-oriented builder for operational applications, and is the fastest way to turn objects, links, and actions into an interactive workflow.
- [OSDK React applications](#) let you build completely custom user interfaces with React, powered by the [Ontology SDK \(OSDK\)](#), when you need an experience beyond what Foundry's built-in tools offer. [Pilot](#) provides a streamlined, AI-powered way to build these OSDK applications from a natural-language prompt.
- [Custom widgets](#) let technical builders extend Workshop with custom frontend code, adding tailored functionality without building a standalone application from scratch.
- [Slate](#) is a low-code, drag-and-drop builder for operational applications, interactive dashboards, and custom landing pages, with optional customization using HTML, CSS, and JavaScript.
- [Code Workspaces applications](#) let you build and publish interactive Streamlit, Dash, and Shiny® applications from third-party IDEs, backed by Foundry data and governance.

Workshop

Workshop is a flexible, object-oriented application building tool. Workshop leverages the semantic primitives (such as objects and links) and the kinetic primitives (such as actions and functions) within the [Ontology](#) to enable the rapid delivery of interactive web and mobile applications. The application building experience in Workshop empowers users to create powerful applications out of no-code, low-code, and code-based widgets. No technical expertise is required to start building with widgets and weaving objects, links, and actions together into user-driven workflows that go [far beyond dashboards](#) or passive visualizations. Meanwhile, code-based enrichment with [functions](#) can be seamlessly embedded within Workshop widgets to allow for complex interactions, cascading processes, and complex data capture. When a workflow needs functionality beyond the built-in widgets, you can extend a Workshop application with [custom widgets](#) to add tailored, code-based components without leaving Workshop.

[Learn more about Workshop.](#)

OSDK React applications

OSDK React applications let you build completely customizable user interfaces using [React](#), powered by the [Ontology SDK \(OSDK\)](#) and created through [Pilot](#). By treating Foundry as your backend, you can combine the React ecosystem with Ontology's high-scale queries, edits, and granular governance controls to securely deliver

API Reference ↗

Send feedback

bespoke applications. To build an OSDK React application without starting from code, [Pilot](#) offers a streamlined, AI-powered experience that generates the Ontology entities, design, and frontend from a natural-language prompt and guides you through deployment.

[Learn more about OSDK React applications.](#)

Custom widgets

Custom widgets allow technical builders to securely extend Workshop applications with custom frontend code. Rather than building a standalone application from scratch, you can implement functionality that is not available out of the box, such as bespoke visualizations or discipline-specific views of Ontology objects, and embed them directly within a Workshop module. You can also embed a single, full-page custom widget within Workshop to bring the flexibility of custom code to the Workshop framework.

[Learn more about custom widgets.](#)

Slate

Slate provides builders with a flexible set of tools to quickly create operational applications and interactive dashboards. Slate enables application developers to construct dynamic and responsive applications with a drag-and-drop interface, reducing development time and cost. Slate includes capabilities that are seamlessly integrated with the Foundry Ontology, but also enables developers to fully customize applications using HTML, CSS, and JavaScript.

[Learn more about Slate.](#)

Code Workspaces applications

Code Workspaces applications let you build and publish interactive web applications, using [Streamlit](#) or [Dash](#) for Python and [Shiny®](#) for R, directly from third-party IDEs in [Cloud IDEs](#). Data scientists and developers can turn analyses into operational applications on Foundry data, hosted in Foundry containers with the platform's version control, branching, and data governance built in.

[API Reference](#)

Send feedback

[Learn more about Code Workspaces.](#)

Workflow building and management

The primary workflow building and management tools in the Palantir platform are [Workflow Lineage](#), [Automate](#), [Solution Designer](#), and [Use Cases](#).

Workflow Lineage

Workflow Lineage provides an interactive workspace for understanding and managing applications and their underlying processes. With Workflow Lineage, you can explore workflows and view details on objects, actions, functions, large language models, and applications. Workflow Lineage is particularly useful for application builders that are creating, debugging or maintaining workflows. The graph of provenance, deeper property and workshop widget/variable provenance, and upgrade tooling are all helpful when making changes to or extending a workflow.

Automate

Automate gives you a single entry point for setting up and executing all business automation in the platform. The Automate application allows users to define conditions and effects; conditions are checked continuously, and effects are executed automatically when the specified conditions were met.

[Learn more about Automate.](#)

Carbon

Carbon enables the configuration of tailored platform experiences, known as workspaces, for specific user groups. Carbon can provide a focused experience for less technical users that need to carry out critical operational workflows. Each Carbon workspace is a curated collection of applications and resources that can be configured to optimize a given set of operational, end-user workflows. For example, an aircraft parts maintenance workflow might consist of a Workshop application containing a dynamically updated list of parts requiring maintenance, along with Ontology-driven Actions for triaging each part, another

[API Reference](#) ↗

Send feedback

application that is used to investigate each part's maintenance issue; and a Quiver analysis showing maintenance trends over time. Carbon allows the rich tapestry of Foundry applications and analytical capabilities to be integrated into focused, operational experiences.

[Learn more about Carbon.](#)

Solution Designer

Solution Designer is an interactive tool for creating architectural representations of solutions built using the Palantir platform, including representations for first and third-party integration points, links to platform resources, on-demand access to documentation and best practices, and more.

[Learn more about Solution Designer.](#)

Use Cases

The Use Cases application allows builders to organize their work within a single operational interface. By combining the file system view with an ontology management view, developers can access a curated view focused on the work for which they are responsible.

[Learn more about the Use Cases application.](#)

Developer toolchain

The Palantir [developer toolchain](#) enables you to build your own applications on top of the Palantir platform using your own tools.

The centerpiece of the Palantir developer toolchain is the [Ontology SDK \(OSDK\)](#), generated in [Developer Console](#). The Ontology SDK is created either as an npm package for TypeScript or as a pip or Conda package for Python, and it only contains a pre-selected subset of your C API Reference ↗. It lets you access object types, apply actions to update data in the Ontology, call functions, and run AIP Logic functions for [AIP-enabled](#) enrollments. The

Developer Console also includes Ontology-specific documentation for

Send feedback

for your application. Applications use the OAuth flow as a public or confidential client to access the data.

[Learn more about Ontology SDK.](#)

You can also work directly with Foundry's [REST APIs](#) to query objects, apply actions, and call functions against the Ontology from any language or runtime for applications built entirely outside of Foundry.

NEXT
What is an operational application? →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Use case development / What is an operational application?

What is an operational application?

At Palantir, we frequently refer to workflows and applications as being "operational." An **operational application** is a user interface designed to improve organizational decision-making. A traditional dashboard or report delivers read-only information and stops there; an operational application enriches this information with simulations and actions to ensure human users can make optimal decisions with ease.

An **operational application** is a user interface designed to improve organizational decision-making. A traditional dashboard or report delivers read-only information and stops there; an operational application enriches this information with simulations and actions to ensure human users can make optimal decisions with ease. Generally, workflows that drive decision-making are far more likely to gain user adoption and promote organizational outcomes than those that only present information. The guide below describes what makes applications operational and how to use Foundry's application-building capabilities to create them in practice.

Beyond the dashboard: Elevating an idea into an operational application

Most application ideas begin as something read-only: a chart, a summary table, or a report that a team checks regularly. This is a useful starting point, but it leaves the most valuable step to the user, who must interpret the information, decide what to do, and then take action with another tool. An operational application closes that gap. You can elevate almost any read-only idea by working through the following steps.

[API Reference ↗](#)

1   with the decision, not the screen

[Send feedback](#)

Before building anything, name the decision or decisions the application exists to support. Ask what choice the user is there to make, what information they need to make it well, and what happens once they decide on a course of action. Anchoring on the decision keeps the application focused on organizational outcomes and clarifies downstream design choices.

2. Surface the right information at the right time

Once you know the decision, identify what information the user must have to make it. Because operational applications read live from the [Ontology](#), you can surface the current state of the world at the moment a decision needs attention rather than a stale snapshot. Show only the objects, links, and properties that matter to the decision, along with context needed to act, and leave other information out. This focus gives the application a clear purpose rather than presenting the entire contents of a source system in an information-dense, hard-to-navigate interface.

3. Test the impact of a decision before committing

A true operational application does not only present a decision; it lets users explore the consequences of that decision before they commit to it. Using [scenarios](#), a user can fork the Ontology, apply one or more [actions](#), and see how the world would look under each option without writing anything back to the live system. When a scenario incorporates [models](#), a simulation can predict complex downstream effects and allow the user to weigh tradeoffs and compare competing plans of action. This approach of combining human expertise with high-quality data and rigorous modeling can achieve better organizational outcomes than acting on intuition or unverified assumptions alone.

4. Turn information into action

The defining feature of an operational application is that the user can make a business-critical decision *within that same application* rather than in another tool. In the Ontology, [actions](#) are the governed mechanism that makes this possible: they define how users write their decisions back into the system. Action [submission criteria](#) ensure that the Ontology c

API Reference ↗

ordance with your organization's rules. This framework lets users act on a decision the moment they reach it without the friction of navigating multiple interfaces.

Send feedback

5. Close the loop

When a user submits an action in Foundry, their decision is committed as an atomic change in the Ontology and is immediately reflected across other applications to every other user. One user's decision is not isolated in a personal spreadsheet or email thread; it immediately becomes part of the shared, real-time view of the world that everyone else uses to make their own decisions. This is one way the Ontology compounds the value of decisions across the organization over time rather than being locked within departmental silos.

When a user makes a decision via an Ontology action, it can also drive downstream effects:

- [Side effects](#) can send notifications, including by email, or trigger events in external systems. For example, [webhooks](#) let an action call external APIs or other Foundry APIs, so a decision captured in Foundry can update a system of record, post a message outside the platform, or kick off another process.
- [Function-backed actions](#) let you define action types of arbitrary complexity, using code to determine how objects should change, whether that logic runs as a serverless function or as a [compute module](#).
- [Triggering schedule builds](#) allows users to start data integration builds.
- [Automate](#) turns a captured decision into ongoing business automation, checking conditions continuously or on a schedule and running effects automatically when those conditions are met. [Autopilot](#) then gives teams a control center to visualize, monitor, and troubleshoot these automation workflows at scale.
- Actions can trigger [agents](#) to carry out follow-on work, or route an agent's proposed changes to a person for approval before they are applied, as described in [Extending the loop with agents](#) below.
- The [action log](#) records every action submission as an object type in the Ontology, turning the history of decisions and their rationale into data you can analyze, audit, and feed into other decision-making workflows.

The Ontology allows decisions in operational applications to become powerful points of leverage, triggering downstream processes and creating a durable, auditable record of organizational choices. When decisions themselves become part of the Ontology, they can inform future choices and compound into a repeating cycle of increasingly better outcomes.

API Reference ↗

A real-world example

Send feedback

Consider a logistics coordinator at a company that delivers parts to automotive assembly plants just in time, where a single late shipment can idle a production line and trigger steep penalties. A shipment of transmission components is delayed by a carrier breakdown and is now at risk of missing the plant's delivery window.

With a traditional dashboard

- The dashboard flags the shipment, but the coordinator must leave it to assemble the picture by hand: a spreadsheet of approved carriers, emails asking about capacity, a separate system for the customer's window and penalty terms, and transit times estimated from memory.
- Under time pressure and with incomplete information, they book a carrier that seems reasonable over the phone.
- The decision lives nowhere others can see: the planning team does not know the route changed, the customer is not notified, and a colleague restarts the same investigation an hour later.
- Nothing is recorded, so the reasoning is lost and cannot be audited or learned from. Tomorrow's process looks exactly like yesterday's process.

With an operational application

- The application opens on the decision itself and surfaces only what it requires, live from the Ontology: the parts on board, the delivery window and penalty clause, the current carrier, and alternative carriers with real-time capacity, cost, and on-time history.
- The coordinator simulates each candidate carrier in a scenario, where models project arrival time, cost, on-time probability, and knock-on effects to other shipments, then commits to the option with the best overall outcome.
- A single action reassigns the carrier, commits the decision to the Ontology, notifies the customer, and updates the planning pipeline at once, without leaving the application.
- Everyone running a parallel process sees it immediately, each in their own application: customer service has the revised estimate and its reason, finance picks up the new cost and the penalty avoided, and the plant's planner adjusts the build schedule. No one had to be notified.

API Reference ↗

and its rationale becomes durable, auditable data. Teams can compare each scenario's expected outcome against what actually happened so each decision makes the next one better.

Send feedback

Extending the loop with agents

Operational applications only become more important as you develop automated, agentic processes. Deploying [AI agents](#) into workflows does not replace the decision-making loop; it allows human users to operate across a broader decision-making plane. Users apply judgment at critical points that require nuanced human expertise and set high-level direction for agents and agent swarms (groups of agents working in parallel) to carry out routine work that can be automated. The operational application remains essential in this framing: it is the interface where human users review, steer, and approve the work delegated to agentic processes, keeping a person accountable for the decisions that matter most.

Return to the logistics example. The accumulated record of past decisions is what makes the workflow ready for agents:

- Because that record captures both the quantitative outcomes and the coordinator's qualitative rationale, an agent can take on a subset of the reassignments itself.
- The coordinator sets the criteria for what the agent may handle, such as lower-value shipments on well-traveled routes where one carrier clearly wins on cost, arrival time, and on-time probability. Anything ambiguous or high-stakes is escalated back to a person.
- The coordinator shifts from making every reassignment to supervising the automated ones, using the same application to monitor performance, watch for drift, and adjust guardrails.
- The coordinator still personally handle the difficult, context-heavy rerouting decisions that depend on experience and judgment, and the record continues to compound.

Signs you have built a dashboard, not an operational application

Use the following checks to assess whether a user interface is an operational application:

- **No decision is named:** If you cannot state a specific decision users will be making in the [API Reference ↗](#), it's likely still an informational dashboard.

Nothing is written back: If the application only reads data and never captures a decision, it cannot compound value over time. It is using the Onto database, not as a representation of the world.

Send feedback

- **The user has to act somewhere else:** If the user gathers information from the application but then has to trigger decisions in another system, the application is not operational.
- **Alternatives cannot be compared:** If the user cannot simulate different scenarios and weigh the tradeoffs of different decisions before committing to one, the application forces blind decisions.
- **Decisions stay local:** If a captured decision is not reflected back to everyone working with the same data, teams fall out of sync and the decision cannot compound.

Building operational applications in Foundry

Foundry offers several tools for building operational applications, spanning from no-code options to fully custom processes:

- [Workshop](#): The primary no-code tool for building object-oriented operational applications, with interactive layouts, events, and widgets that write decisions back to the Ontology.
- [OSDK React applications](#): Fully custom React interfaces powered by the Ontology SDK for custom experiences beyond Foundry's built-in tools. [Pilot](#) is a streamlined, AI-powered way to build these OSDK applications from a natural-language prompt.
- [Custom widgets](#): Extend Workshop with bespoke frontend React code, such as for tailored visualizations, without building a standalone application. Note that you can embed a single, full-page custom widget within a Workshop module to leverage the capabilities of both tools.
- [Slate](#): A builder for custom applications and dashboards using HTML and JavaScript on Ontology data.
- [Code Workspaces applications](#): Interactive Streamlit, Dash, and Shiny® applications built in your preferred IDE and published on Foundry.
- [REST APIs](#): Build applications against the Ontology entirely outside of Foundry's frameworks.

Get started by building a simple application in [Workshop](#) to get familiar with application-building principles with the Ontology.

API Reference ↗

← Overview

Connecting an

NEXT →

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Use case development / Connecting analytics to operations

Connecting analytics to operations

Although Foundry's focus is upon enabling [operational workflows](#) that capture data back into the system, the results of work done using Foundry's [analytical capabilities](#) is also a key part of the puzzle. This page covers how analytical tools can be used to deliver value to end users in a cohesive way that empowers decision-making.

Dashboards

Often, the results of open-ended exploration of data in the Ontology can result in interesting insights that could be useful for other people in your organization. Starting with a set of objects, drilling down into a subset, and exploring properties using charts or linked objects may yield a repeatable workflow that can be combined with other platform capabilities in a valuable way.

[Quiver](#) is the application used for open-ended analysis on object data in Foundry. Quiver dashboards enable you to create parameterized, curated views that end users can use to explore data in a structured fashion. Once you've published a dashboard, you can use it in operational applications in a few different ways:

- Quiver dashboards can be [embedded into Workshop applications](#), allowing you to flexibly show dashboards to users as part of a structured workflow.
- Quiver dashboards can be embedded into [Carbon](#) so that users can access the template as a tab in their workspace.

Using this functionality, exploratory analysis can quickly transition from an ad-hoc dashboard to being a core part of an operational workflow.

API Reference ↗

Foundry's [machine learning](#) capabilities provide a full suite of tools for model development, integration, evaluation, and deployment. Once models have been validated according to organizational criteria, they can be deployed to end users easily.

Workshop's support for [Scenarios](#) enables the deployment of machine learning directly to end users without requiring any custom development or extensive configuration. Users interact with concepts they are familiar with, while seeing model-backed results in an easy-to-understand form such as a *forecast* or *estimate*.

For more details on tying machine learning to organizational outcomes, refer to [Models in the Ontology](#).

PREVIOUS

← [What is an operational application?](#)

NEXT

[Conduct sentiment analysis with AIP](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

—[Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Use case development / Conduct sentiment analysis with AIP

Conduct sentiment analysis with AIP

Sentiment analysis is a transformative approach in data-driven decision-making that has revolutionized how businesses understand customer opinions, market trends, and brand perception. This guide delves into the intricacies of sentiment analysis and explores how advanced analytics offerings like Palantir's AIP (Artificial Intelligence Platform) can enhance this process.

Sentiment analysis represents a complex domain at the nexus of data science and psychology, designed to interpret the broad array of human emotions embedded in textual content. By analyzing data sources ranging from tweets to reviews, sentiment analysis can provide deep insights into consumer behavior and societal trends which businesses can leverage to inform strategic decisions, optimize product offerings, and tailor marketing campaigns for maximum impact.

Why sentiment analysis matters

Transcending academic interest, sentiment analysis offers concrete, actionable intelligence serving multiple purposes across industries in diverse sectors by converting subjective opinions into quantifiable data; The outputs of which can thereby shape marketing, product development, and customer service.

- **Customer feedback analysis:** Firms scrutinize customer reviews to enhance product quality and service delivery.

API Reference ↗

Sentiment analysis deciphers market dynamics and consumer

insights from digital content.

Send feedback

- **Financial markets:** Market participants predict trends by analyzing sentiment in financial discourse.

Traditional approach to sentiment analysis

The following describes the tools and techniques with which sentiment analysis is typically performed.

Preprocess the data

Traditional sentiment analysis requires extensive manual preprocessing to refine text data, a process critical yet labor-intensive for NLP model efficacy. Key preprocessing tasks include:

1. **Tokenization:** Breaking down text into meaningful units (tokens), such as words or symbols. For instance, "I love apple pies!" becomes ["I", "love", "apple", "pies", "!"]. Transforming text into tokens demands meticulous rules to handle nuances in different languages or contexts.
2. **Stop word removal:** Eliminating common but low-value words ("is", "and", "the") to focus analysis. After removal, ["I", "love", "apple", "pies", "!"] transforms into ["love", "apple", "pies"].
3. **Stemming and lemmatization:** Reducing words to their base or root form for consistent analysis. "Loving" is simplified to "love". These methods often struggle with irregular word forms, leading to inaccuracies in text interpretation.

Rule-based systems

These systems apply a set of predetermined rules crafted by linguistic experts. For example, if a text contains more positive words from a predefined list, it is classified as positive. They are heavily reliant on comprehensive, manually curated linguistic rules, and struggle to adapt to context, sarcasm, and subtleties in language.

[API Reference](#) [Machine learning systems](#)

[Send feedback](#)

1. **Supervised learning:** Models learn from labeled data, associating text features with sentiment labels. Example: Naive Bayes classifiers predicting sentiment from product reviews. However, supervised learning relies on large, well-annotated datasets for each task, which are expensive and time-consuming to create. Models created through supervised learning may also suffer from bias in training data, potentially skewing sentiment analysis results.
2. **Unsupervised learning:** Identify patterns and sentiments without explicit labels. Example: Discovering sentiment clusters in tweets using k-means clustering. Unsupervised learning faces challenges in accurately discerning sentiment without clear labels, leading to ambiguous outcomes. Sensitivity to data quality and noise can also result in unreliable sentiment clusters.
3. **Deep learning in sentiment analysis:**
 1. Convolutional Neural Networks (CNNs): Effective for sentence classification by identifying key phrases. While adept at pattern recognition, they may overlook the sequential nature of text.
 2. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM): Ideal for analyzing sequential data, such as longer texts. Can be computationally-intensive and prone to overfitting on smaller datasets.
 3. Transformers and BERT: These are medium-sized models that require fine-tuning to adhere to the sentiment analysis task, which requires substantial computational resources, datasets and expertise to fine-tune.

Popular tools and libraries

Several tools and libraries have become staples in the sentiment analysis landscape, each offering unique features and capabilities:

1. [NLTK \(Natural Language Toolkit\)](#) ↗: Offers extensive resources for text processing, ideal for research and education. While rich in resources, it may lack the sophistication needed for complex analysis.
2. [TextBlob](#) ↗: Provides a simple API for basic NLP tasks, suitable for prototyping and small projects. Its simplicity is a double-edged sword, limiting advanced customization and scalability.
3. [Stanford CoreNLP](#) ↗: Delivers tools for advanced NLP tasks, known for its accuracy in

API Reference ↗

 and named entity recognition. Offers powerful tools but at the cost of computational efficiency and ease of use.

Send feedback

4. [spaCy 7](#): Focuses on speed and efficiency, excelling in large-scale information extraction and deep learning tasks. Requires significant effort to tailor for specific sentiment analysis tasks.

Traditional sentiment analysis: Inherent technical challenges

1. **Sarcasm and irony detection:** Conventional models lack the sophistication to discern such subtleties, necessitating advanced contextual comprehension.
2. **Contextual polarity:** Algorithms struggle with the fluidity of word meanings influenced by context, complicating sentiment determination.
3. **Language and cultural variations:** Heterogeneity in sentiment expression across cultures and languages impedes standardized analysis.
4. **Manual preprocessing:** Laborious and rule-intensive data preparation processes are required, increasing the potential for error and inefficiency.
5. **Data dependency:** Automatic systems rely on vast, annotated datasets, which are costly and laborious to produce, with inherent biases.
6. **Computational demands:** Deep learning techniques, while powerful, necessitate significant computational resources and expertise.
7. **Adaptability:** Rule-based and hybrid systems lack the agility to evolve with language use and are often rigid in their application.
8. **Tool limitations:** Existing NLP tools and libraries may not offer the necessary depth or flexibility for complex sentiment analysis tasks.

Leverage LLMs for superior sentiment analysis

LLMs such as GPT-4 and Llama offer advanced capabilities for sentiment analysis:

- **Enhanced contextual interpretation:** LLMs excel in discerning sentiment within intricate contexts, surpassing traditional models in identifying nuanced emotional expressions.

API Reference ↗

• **Irony Recognition:** Their sophisticated language models enable LLMs to detect and interpret non-literal language.

- **Multilingual and cross-cultural competence:** With training on diverse datasets, LLMs demonstrate an expanded understanding of sentiment across various

Send feedback

languages and cultural contexts.

Implement LLMs for sentiment analysis

The implementation of LLMs in sentiment analysis involves strategic considerations:

- **LLM selection:** With a wide-range of LLM options available today, an ideal implementation would have the ability to swap out models to find the best fit for your sentiment analysis needs.
- **Prompt engineering:** Prompt engineering involves crafting prompts that effectively harness the LLMs capabilities to deduce sentiment from text.
- **Prompt iteration:** Prompt iteration entails refining prompts iteratively, using trial runs to optimize the model's sentiment analysis accuracy.

Types of LLM-prompted sentiment analysis

1. **Polarity sentiment analysis:** Classifies text into negative, neutral, or positive categories.
2. **Emotional sentiment analysis:** Goes beyond polarity to categorize text according to specific emotions like happiness, sadness, or anger.
3. **Numerical sentiment analysis:** Assigns a bounded numerical value to the sentiment, providing a quantifiable measure of sentiment intensity.

Palantir's approach to sentiment analysis using Pipeline Builder

You can perform enterprise-grade sentiment analysis with Pipeline Builder in the Palantir platform, and revolutionize sentiment analysis deployment through the [Use LLM node](#), streamlining LLM integration into data workflows for scalable, code-minimal sentiment analysis.

With our [Use LLM node](#) tool, you can benefit from:

API Reference ↗

- **Expert-designed prompt templates:** Includes a suite of five templates for sentiment analysis, with a dedicated sentiment analysis option, crafted by prompt engineering experts.

Send feedback

- **Prompt testing capability:** Enables prompt efficacy testing on data subsets, facilitating prompt refinement without necessitating full dataset processing.
- **Effortless data pipeline integration:** The node seamlessly incorporates into existing workflows, introducing advanced LLM processing with minimal disruption.

The Pipeline Builder application and its Use LLM node represent a pivotal advancement in sentiment analysis, offering a scalable, user-friendly platform for leveraging LLMs in extracting sentiment insights. Coupled with the Ontology, Palantir's AIP has become essential for organizations aiming to tap into the depth of public sentiment and emotion.

Build with AIP initiative to enhance your sentiment analysis

Using the Build with AIP application, you can access Palantir's toolkit for common data tasks, including a sentiment analysis starter pack for Pipeline Builder.

Our sentiment analysis starter pack for Pipeline Builder includes:

- **Accelerated sentiment classification:** Swiftly assign sentiment labels (positive, negative, neutral) to textual data.
- **Automated prompt generation:** Streamline prompt crafting, automatically producing comprehensive prompts from user inputs.
- **Real-time LLM testing:** Facilitate live data prompt testing, ensuring result reliability prior to extensive deployment.
- **Optimized data processing:** Incorporate features like rate limiting and error handling for efficient, high-quality data analysis.

Practical applications

Using the starter pack, you can perform the following:

- **Customer feedback insights:** Leverages sentiment analysis to refine products and services based on customer input.
- **Organizational feedback evaluation:** Uses employee feedback for organizational improvement insights.

API Reference ↗

Analysis: Automates sentiment sorting in product reviews to highlight improvement opportunities.

Send feedback

- **Content moderation:** Employs sentiment tracking for proactive content management, safeguarding brand and community integrity.

PREVIOUS

← [Connecting analytics to operations](#)

NEXT

[Curating apps in Applications Portal](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

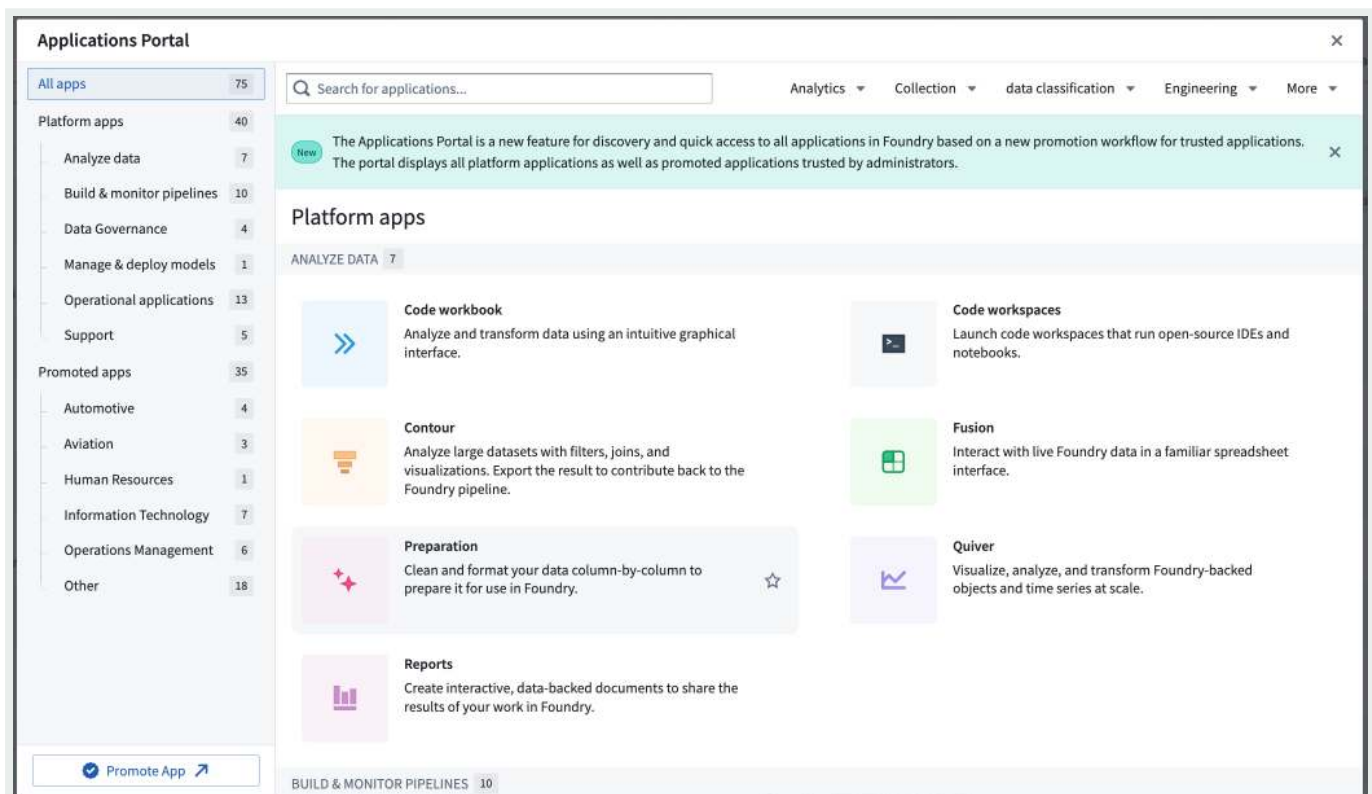
[API Reference ↗](#)

[Send feedback](#)

Use case development / Curating apps in Applications Portal

Curating apps in Applications Portal

Applications Portal is a tool for discovering and accessing all apps in Foundry. This includes both (1) core Foundry platform apps and (2) trusted custom apps that admins promote to Applications Portal.



Navigate to Applications Portal

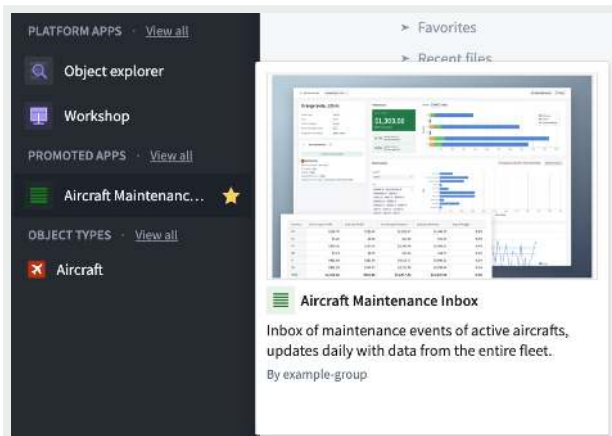
You can open Applications Portal by selecting the **Applications Portal** icon in the left sidebar. You will be able to view any application built in [Workshop](#), [Slate](#), or [Carbon](#) that you have access to. You will also have quick access to all Foundry platform apps.

API Reference ↗

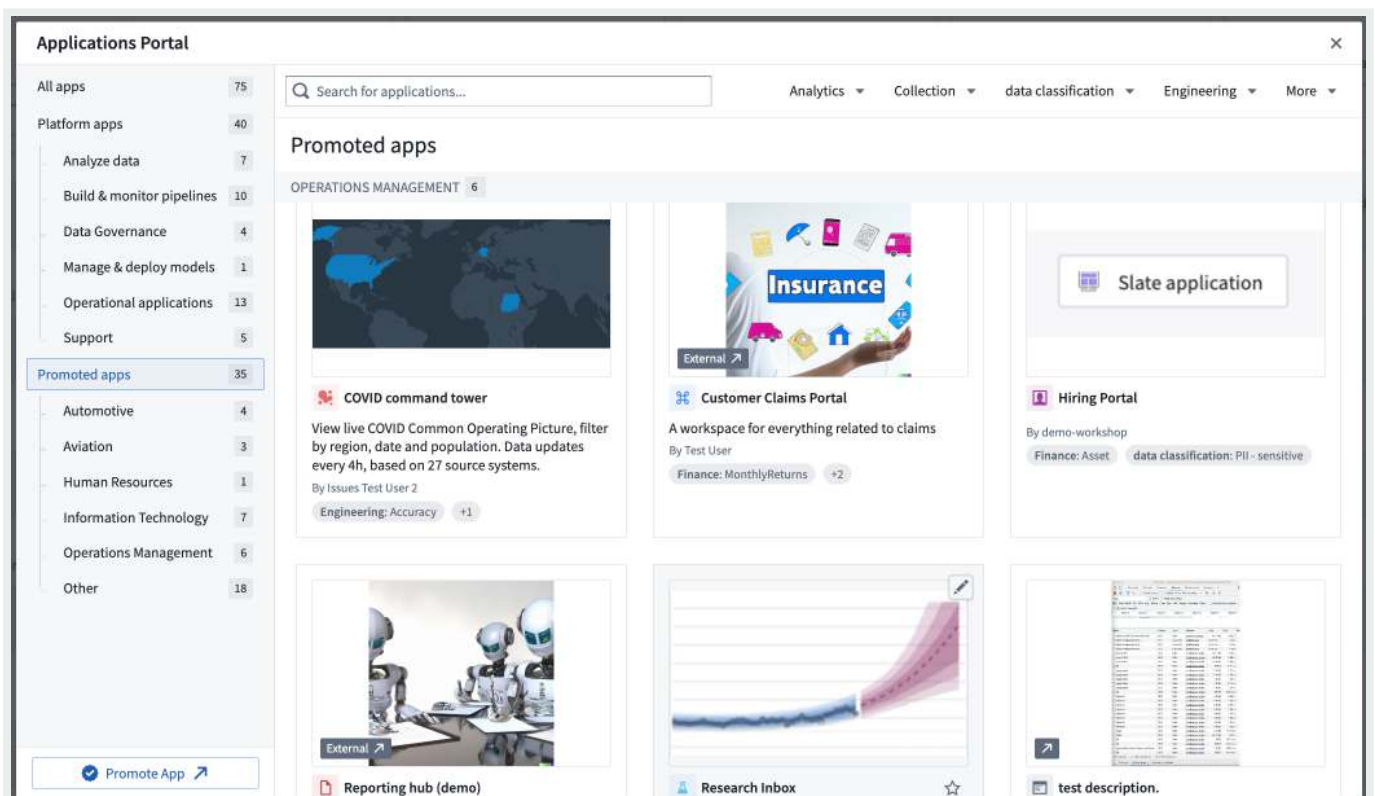
You can pin your favorite apps to the left sidebar for easy access. Select the pin icon next to the application's name from the Applications Portal or when you're editing the

Send feedback

application. Once pinned, a section called **Promoted Apps** will appear in the Foundry sidebar with a list of your favorite promoted apps.



Permissions for Applications Portal are based on [filesystem permissions](#). If you have the Discoverer role, you will be prompted to [request access](#).



API Reference ↗ Portal modes

Applications Portal has two modes of engagement:

Send feedback

- Basic, out-of-the-box: a tool to discover and learn about all Foundry platform apps. No admin work is required. Applications portal is an integral tool in the left sidebar, even without any promoted custom apps.
- Advanced, curation-based: an admin, Palantir representative, or customer governance team promotes all trusted custom apps to Applications Portal.

i There is a single Applications Portal for each enrollment or tenant on a multi-tenant enrollment. The promoted apps that a user can view are limited to the [spaces](#) that belong to their [Organization](#). If you have access to a shared space, you will also see promoted apps from that space.

Promoted apps in Applications Portal

A promoted application is an application that was marked by an admin as trusted and production-ready. Every promotion application has the following required metadata that will be displayed in Applications Portal and in the left sidebar:

- Name: this can be different than the resource name
- Icon
- Description (optional)
- Application Owner
- Thumbnail
- Collections and tags: you can use these to categorize promoted apps in different sections and apply filters in Applications Portal. Collections are required, while tags are optional.

Promoted apps receive the purple checkmark for trusted content, similar to items in the Data Catalog.

Permissions for promoting apps to Applications Portal

API Reference ↗ Workshop modules, Slate applications, Carbon workspaces, or external web links to Applications Portal. Permission to promote apps to Applications Portal is granted in [Control Panel](#). You must have either the “Organization Admin” or “Applications Portal Admin” role.

Send feedback

Experience Administrator” role. You also must be an editor or owner of an app to be able to promote it.

Promoted applications appear in Applications Portal for all users with at least the Discoverer role for the application and its space.

How to promote apps to Applications Portal

The apps promotion UI is available both in Applications Portal and in edit mode of Workshop and Slate (see example below).

Un-promoting a promoted application is also done using the promotion UI, by selecting the **Unpromote** button at the bottom left.

You can change the resource that a promotion references to release new applications in a controlled manner.

[API Reference ↗](#)

[Send feedback](#)

Edit promotion ⓘ

Name

Aircraft Maintenance Inbox

Icon
Appears on the sidebar when an app is pinned

Airplane
Blue 4

Description

Inbox of maintenance events of active aircrafts, updates daily with data from the entire fleet.

Application Owner
User or group responsible for the application. Optional.

example-group

App portal preview

Collections

Choose a collection...
New Collection

Aviation
Operations Management

Tags

Search for tags

data classification: top secret
Engineering: Assembly
Engineering: Kinetics

App portal permission

App portal permission reflects file and project permission. To edit the permission of the file and app portal, please go to the project setting.

Unpromote
Cancel
Update Promotion

You can use collections and tags to categorize apps in Applications Portal and apply filters to only view certain apps.

Permissions to create collections and tags or add them to a promoted app are the same as the permissions to promote to Applications Portal, based on the roles of “Organization administrator” and “User experience administrator” in Control Panel.

Collections

Collections are displayed on the left sidebar as titles of sections in Applications Portal. They are used as the top-line category in Applications Portal to curate your experience.

Collections are created and managed in the Data Catalog. Once you create a collection, you can add a promoted app to it from Data Catalog and from the filesystem.

Only collections that have promoted apps linked to them are displayed in Applications Portal, and only if you have access to view/discover these apps. If a collection has no promoted apps linked to it, or if you do not have access to any apps on the collection, it will not be displayed.

Tags

Tags are displayed as labels on the promoted apps cards and can be filtered from the top right of Applications Portal. You can create and manage tags from the [Tags section of Platform Settings](#). Once they are created, they can be added to a promoted app on the promotion UI, as well as in the filesystem.

Only tags that have promoted apps linked to them are displayed in Applications Portal as filters, and only if the user has access to view/discover these apps. If a tag is not added to any promoted apps, or if a user has no access to any apps with that tag, it would not be displayed.

Managing platform apps in Applications Portal

API Reference ↗

Foundry platform apps are tools like Quiver, Contour, Data Connection, Pipeline Builder, and more. You can configure the option to display or hide platform apps f

Send feedback

Panel under the **Application access** tab. This allows you to show only certain sub-groups of apps in Applications Portal, as well as in the rest of Foundry.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback